Neural Network Approximation of a Nonlinear Surface

Hadef Mohammed

1 Visualization and Training Setup

In this experiment we train several neural networks to approximate the nonlinear function

$$f(x, y) = \sin(\sqrt{x^2 + y^2}) + 0.5\cos(2x + 2y)$$

using randomly generated points in the interval $[-5, 5]$.

The dataset contains 2000 samples (x, y) and their corresponding function value z. To evaluate the performance of the neural network, we visualize:

- The original surface generated by the function.
- The surface predicted by the neural network.
- The evolution of the loss function during training.

The training process uses the following hyperparameters:

- Learning rate: 0.01
- Number of epochs: 10000
- Batch size: 256

The learning rate controls the step size of the gradient descent optimization. The number of epochs determines how many times the model sees the full dataset during training. The batch size defines the number of samples processed before updating the model parameters.

2 Comparison Between Different Neural Network Architectures

We compare several neural network architectures with different numbers of hidden layers and neurons.

2.1 Network [2,4,4,1]

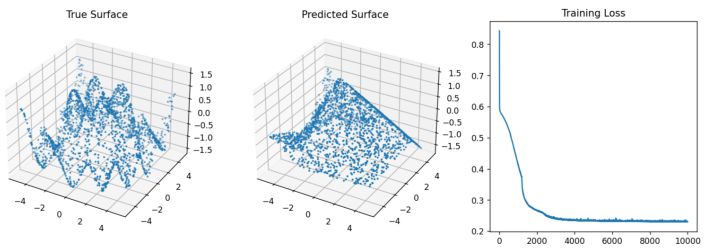


Figure 1: Training result for the neural network architecture [2,4,4,1].

This architecture contains two hidden layers with only four neurons each. Because of its small capacity, the model struggles to approximate the complex nonlinear surface accurately. The predicted surface remains relatively rough and the loss decreases slowly.

2.2 Network [2,64,64,1]

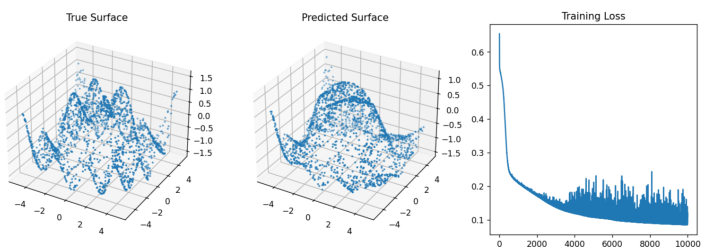


Figure 2: Training result for the neural network architecture [2,64,64,1].

Increasing the number of neurons significantly improves the approximation ability of the network. With 64 neurons per hidden layer, the model can capture more complex patterns of the target function and the predicted surface becomes smoother and closer to the true surface.

2.3 Network [2,64,64,64,1]

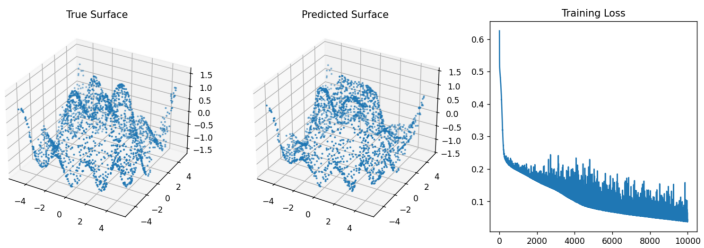


Figure 3: Training result for the neural network architecture [2,64,64,64,1].

Adding an additional hidden layer further increases the representational power of the network. The deeper architecture provides a more accurate approximation of the nonlinear function and leads to a lower training loss.

3 Linear vs Nonlinear Neural Networks

3.1 Training Without Activation Function (Linear Model)

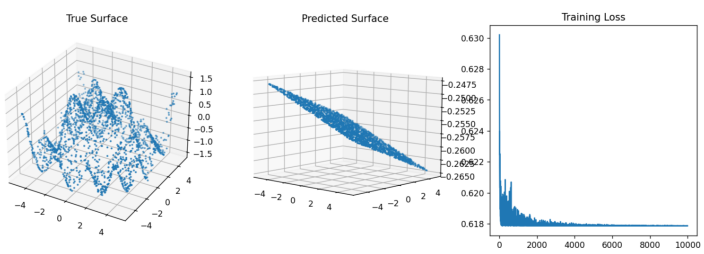


Figure 4: Training result without an activation function.

When no activation function is used, each layer performs only a linear transformation. As a consequence, stacking multiple layers is equivalent to a single linear transformation.

Therefore, the entire neural network behaves like a simple linear regression model:

$$y = Wx + b$$

Since the target function is highly nonlinear, a linear model cannot capture its structure correctly. The predicted surface tends to approximate only a plane and the model fails to represent the oscillatory behavior of the function.

3.2 Why We Use ReLU

To introduce nonlinearity into the network, we use the ReLU (Rectified Linear Unit) activation function:

$$ReLU(x) = \max(0, x)$$

ReLU has several advantages:

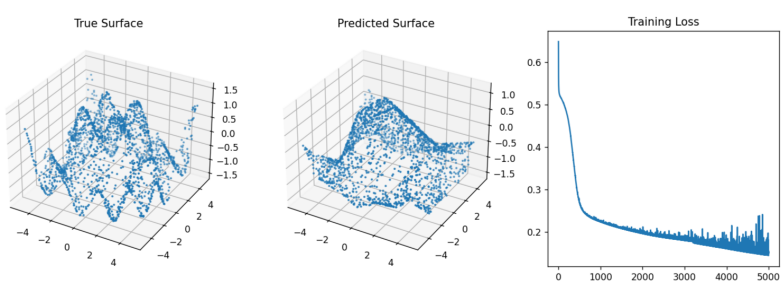
- It introduces nonlinearity, allowing the network to approximate complex functions.
- It is computationally efficient.
- It helps reduce the vanishing gradient problem compared to sigmoid or tanh functions.

Because of these properties, ReLU is widely used in modern neural network architectures.

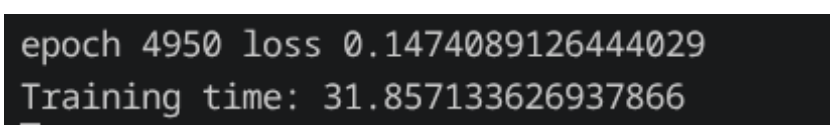
4 Effect of Momentum in Training

Momentum is an optimization technique that accelerates gradient descent by incorporating information from previous gradients.

4.1 Training Without Momentum



(a) Prediction result without momentum



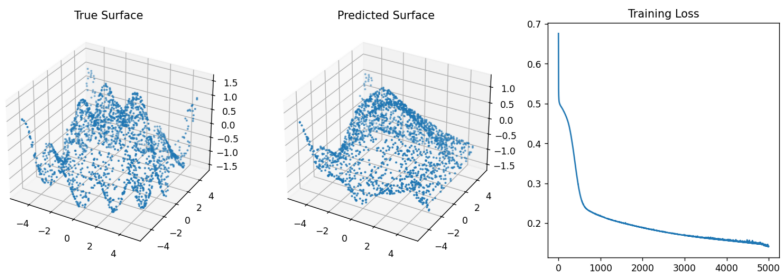
(b) Training time and loss without momentum

Figure 5: Training without momentum

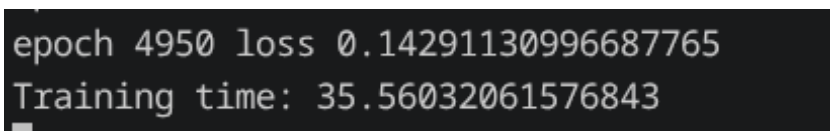
When momentum is not used, the gradient descent updates follow the instantaneous gradient direction. This often leads to oscillations during training, especially in narrow valleys of the loss surface.

As a result, the loss curve tends to exhibit a **zigzag behavior**, indicating unstable updates and slower convergence.

4.2 Training With Momentum



(a) Prediction result with momentum



(b) Training time and loss with momentum

Figure 6: Training with momentum

Momentum reduces oscillations by accumulating gradients from previous iterations. Instead of following the gradient direction strictly at each step, the optimizer maintains a velocity that smooths the updates.

This leads to:

- Faster convergence
- Smoother loss curves
- Reduced zigzag behavior during training

As a result, models trained with momentum generally converge faster and reach lower loss values.

5 Conclusion

This experiment highlights several important aspects of neural network training.

First, increasing the depth and width of the network improves its ability to approximate complex nonlinear functions. Second, activation functions such as ReLU are essential because they introduce nonlinearity into the model. Without them, the network behaves like a simple linear regression model.

Finally, optimization techniques such as momentum significantly improve training stability and convergence speed by reducing oscillations in the optimization process.